

Programa: Análisis y Desarrollo de Software

Actividad: GA4-220501095-AA2-EV05

Desarrollar la arquitectura de software, de acuerdo con el patrón de diseño seleccionado

*Sistema para aprender Armonía Musical con un asistente LLM (IA) - **MusicAI***



Aprendiz: Juan Carlos Carvajal Palmar

ADSO - 3336142

Instructor: Juan Hernández

Servicio Nacional de Aprendizaje - SENA

Fecha Presentación: 17 de mayo de 2026

Introducción

El desarrollo de software moderno requiere arquitecturas sólidas, organizadas y escalables que permitan garantizar la mantenibilidad, reutilización y evolución de los sistemas informáticos a lo largo del tiempo. En este contexto, la arquitectura de software representa uno de los elementos más importantes dentro del ciclo de vida del desarrollo, ya que define la estructura lógica y física del sistema, los componentes que lo conforman, las tecnologías utilizadas y la manera en que interactúan entre sí para cumplir los requisitos funcionales y no funcionales establecidos.

El presente trabajo tiene como objetivo desarrollar la arquitectura de software del proyecto **MusicAI**, aplicando patrones de diseño, principios de modularidad y buenas prácticas de ingeniería de software orientadas a la construcción de una aplicación moderna enfocada en el aprendizaje de armonía musical mediante apoyo de inteligencia artificial.

MusicAI es una plataforma tecnológica orientada principalmente a dispositivos móviles Android, diseñada para apoyar el aprendizaje progresivo de teoría musical, armonía funcional y práctica instrumental, integrando herramientas interactivas, ejercicios personalizados, reconocimiento musical y asistencia inteligente para mejorar la experiencia de aprendizaje del usuario. El sistema busca combinar educación musical, procesamiento digital de señales e inteligencia artificial para ofrecer una experiencia dinámica, práctica y adaptativa.

Dentro del alcance del sistema, **MusicAI** permitirá:

- Gestionar usuarios y autenticación segura.
- Administrar lecciones y contenidos educativos.
- Generar ejercicios teóricos, auditivos y prácticos.
- Evaluar ejercicios mediante reconocimiento musical.
- Analizar señales de audio capturadas desde el micrófono.
- Llevar seguimiento del progreso académico del estudiante.
- Implementar un agente de inteligencia artificial especializado en teoría musical y armonía funcional.
- Proporcionar recomendaciones y ejercicios personalizados según el desempeño del usuario.

Para el desarrollo de esta evidencia se definieron las tecnologías, patrones de diseño y componentes principales que conformarán la arquitectura lógica y física del sistema, estableciendo además los diagramas de componentes y despliegue necesarios para representar la interacción entre módulos, servicios, bases de datos, dispositivos y servidores involucrados en el funcionamiento de **MusicAI**.

Finalmente, este trabajo permite establecer una visión técnica integral del software **MusicAI**, sirviendo como base para futuras etapas de ejecución, desarrollo, implementación y despliegue del sistema.

Desarrollar la arquitectura de software, de acuerdo con el patrón de diseño seleccionado

1. Arquitectura General para el Software *MusicAI*

MusicAI se desarrollará bajo una arquitectura moderna orientada principalmente a dispositivos móviles Android, basada en separación por capas y arquitectura modular, permitiendo dividir el sistema en componentes independientes para facilitar el mantenimiento, escalabilidad y reutilización del software.

La arquitectura estará compuesta por las siguientes capas principales:

Frontend: Encargado de la interfaz gráfica interactiva que permitirá acceder a lecciones, ejercicios, progreso y herramientas de inteligencia artificial mediante una experiencia moderna, dinámica y orientada principalmente a dispositivos móviles Android.

Tecnologías propuestas:

- Flutter
- Dart
- Figma para UI/UX

Backend: Encargado de procesar la lógica de negocio, autenticación, gestión de usuarios, ejercicios, progreso y comunicación entre los diferentes módulos internos del sistema **MusicAI**.

Tecnologías propuestas:

- Python
- FastAPI
- JWT para autenticación

Base de Datos: Responsable del almacenamiento de usuarios, lecciones, ejercicios, resultados y progreso académico.

Tecnología propuesta:

- SQLServer

Módulo de Inteligencia Artificial: Permitirá responder preguntas, explicar conceptos musicales, generar recomendaciones y brindar apoyo interactivo durante el proceso de aprendizaje del estudiante.

El sistema contempla dos posibles enfoques arquitectónicos para la implementación del agente IA:

1) Primera opción: Agente IA Propio

El sistema implementará un agente de inteligencia artificial propio especializado en teoría musical y armonía funcional.

Posibles tecnologías:

- Python
- TensorFlow
- PyTorch
- Procesamiento de Lenguaje Natural (NLP)
- Bases de conocimiento musicales

2) Segunda opción: Integración de asistente IA mediante API:

Uso de servicios de inteligencia artificial externos mediante API, permitiendo integrar modelos avanzados de lenguaje capaces de responder preguntas relacionadas con teoría musical y aprendizaje interactivo.

Tecnologías propuestas:

- OpenAI API
- FastAPI

Módulo de Reconocimiento Musical: Permitirá analizar el audio capturado desde el micrófono para validar ejercicios prácticos ejecutados por el usuario.

Tecnologías propuestas:

- Python
- Librosa
- Procesamiento Digital de Señales (DSP)
- NumPy
- SciPy

Infraestructura y Despliegue: Permitirá ejecutar y distribuir los diferentes servicios y módulos de **MusicAI** mediante una arquitectura híbrida orientada a dispositivos móviles, servidores propios y servicios de procesamiento especializados.

Herramientas propuestas:

- Docker
- GitHub
- APIs REST
- Linux
- Servicios Cloud (opcional)

Flujo General del Sistema

- El usuario accede a **MusicAI** desde la aplicación móvil.
- El frontend envía solicitudes al backend mediante APIs.
- El backend procesa autenticación, ejercicios y lógica del sistema.
- La información se almacena y consulta en SQL Server.
- El módulo IA analiza consultas y genera respuestas, recomendaciones y ejercicios personalizados.
- El módulo de reconocimiento musical analiza ejercicios prácticos mediante micrófono.
- El sistema actualiza el progreso y muestra resultados en tiempo real.

2. Patrones de Diseño Propuestos para *MusicAI*

Con el objetivo de garantizar una arquitectura organizada, mantenible y escalable, el sistema *MusicAI* implementará diferentes patrones de diseño orientados a buenas prácticas de desarrollo de software. Estos patrones permitirán mejorar la estructura del código, reutilizar componentes y facilitar futuras actualizaciones del sistema.

Los principales patrones de diseño propuestos para *MusicAI* son:

MVC (Model View Controller)

El patrón *MVC* permitirá separar la lógica del sistema en tres componentes principales:

- **Modelo:** gestión de datos y lógica de negocio.
- **Vista:** interfaz visual de la aplicación móvil.
- **Controlador:** procesamiento de solicitudes e interacción entre vista y datos.

Este patrón permitirá organizar y desarrollar de forma independiente el frontend, backend y bases de datos.

Aplicación en *MusicAI*:

- Flutter gestionará la vista de usuario.
- FastAPI procesará la lógica del sistema.
- SQL Server almacenará la información académica y de usuarios.

Singleton

El patrón *Singleton* permitirá que determinadas clases tengan una única instancia activa dentro del sistema.

Aplicación en *MusicAI*:

- Conexión a la base de datos
- Autenticación JWT
- Servicios de configuración
- Conexión con el módulo de inteligencia artificial
- Servicios DSP de reconocimiento musical

Esto ayudará a optimizar recursos y evitar múltiples conexiones innecesarias.

Factory Method

El patrón *Factory Method* permitirá crear diferentes tipos de ejercicios de manera dinámica sin depender directamente de clases específicas.

En *MusicAI* con la aplicación de este patrón se podrá generar:

- Ejercicios teóricos
- Ejercicios auditivos
- Ejercicios prácticos con reconocimiento musical
- Ejercicios personalizados según el progreso del usuario

Esto facilitará la escalabilidad y reutilización de los componentes educativos del software.

Observer

El patrón *Observer* permitirá notificar automáticamente cambios importantes dentro del sistema cuando ocurra un evento específico.

Se aplicará en *MusicAI* cuando el usuario complete:

- Una lección

- Un ejercicio
- Una práctica musical

El sistema actualizará automáticamente:

- Progreso
- Estadísticas
- Niveles
- Recomendaciones
- Desbloqueará contenido nuevo

Esto permitirá mantener sincronizada la información en tiempo real.

Strategy

El patrón *Strategy* permitirá utilizar diferentes métodos de validación dependiendo del tipo de ejercicio que esté realizando el usuario.

En **MusicAI** con este patrón el sistema podrá utilizar estrategias distintas para:

- Validación teórica
- Reconocimiento musical mediante DSP
- Análisis de señales de audio
- Evaluación mediante IA
- Ejercicios rápidos, avanzados o personalizados

Esto permitirá que cada actividad tenga una lógica de evaluación independiente y flexible.

Facade

El patrón *Facade* permitirá simplificar procesos complejos del sistema mediante interfaces más sencillas para el usuario.

Cuando el estudiante inicie una práctica musical, el sistema **MusicAI** ejecutará internamente:

- Activación de micrófono
- Carga del ejercicio
- Validación de audio
- Análisis musical
- Actualización de progreso
- Generación de resultados

Sin embargo, el usuario solo visualizará una acción simple desde la interfaz principal.

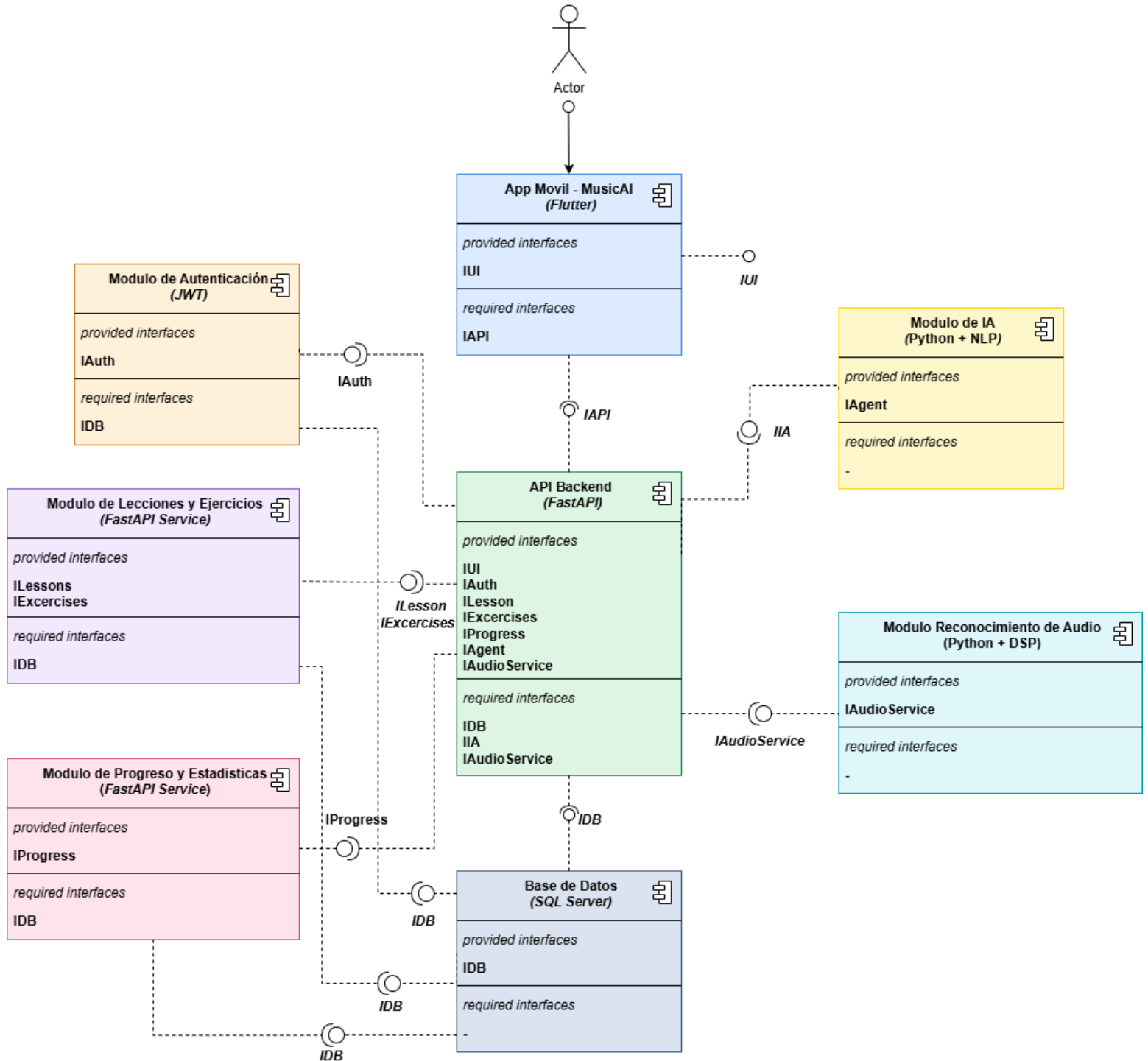
La implementación de estos patrones permitirá que el programa **MusicAI** mantenga una arquitectura moderna, modular y preparada para futuras mejoras, facilitando tanto el desarrollo como el mantenimiento del sistema a largo plazo.

La combinación de arquitectura modular, *APIs REST*, procesamiento digital de señales (*DSP*) y patrones de diseño permitirá que **MusicAI** sea una plataforma escalable, mantenible y preparada para futuras mejoras del agente de inteligencia artificial propio o integraciones de inteligencia artificial vía API, destacando entre las tecnologías de aprendizaje musical inteligente.

3. Diagrama de componentes

Link: [Diagrama de Componentes](#)

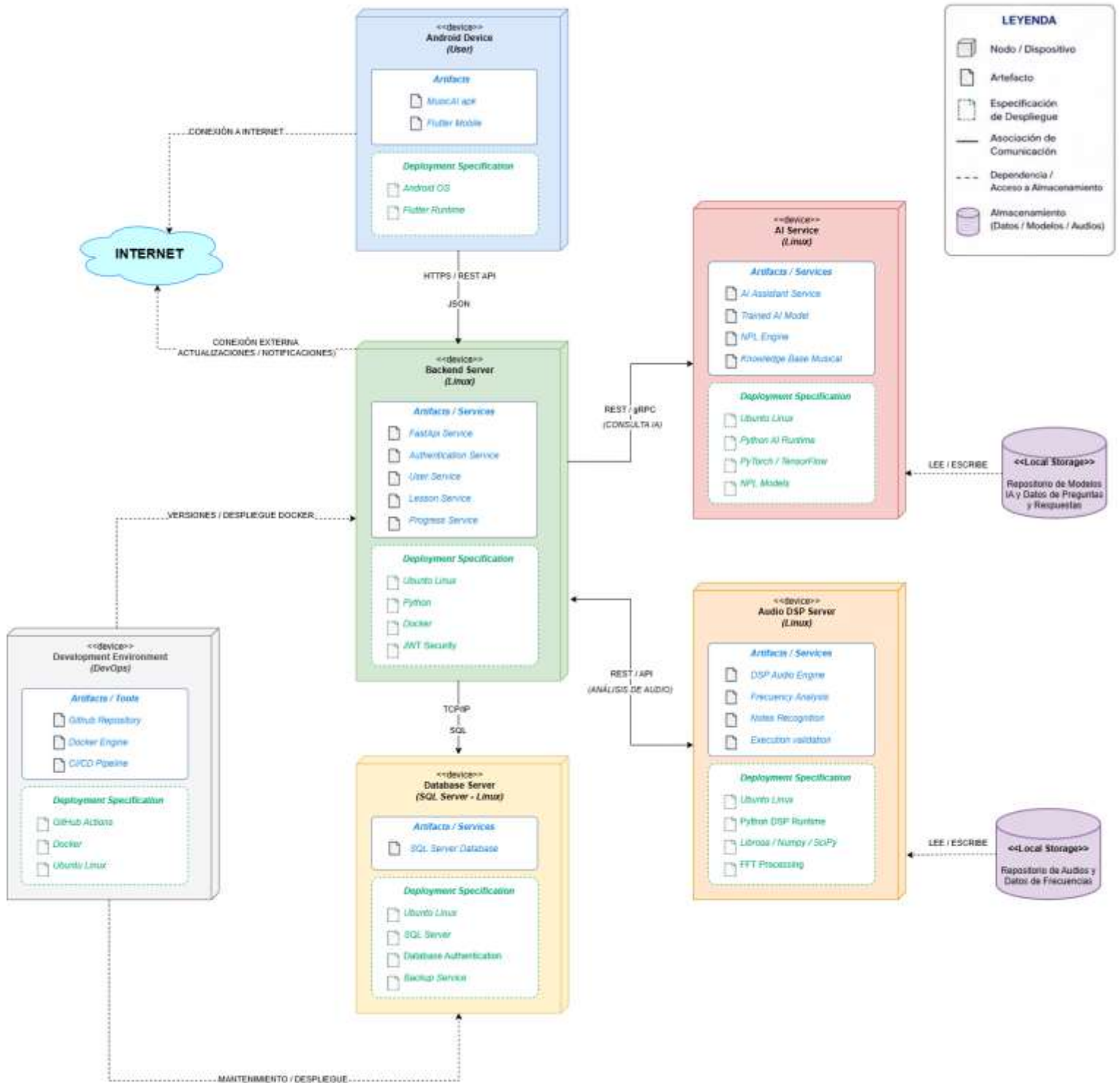
Diagrama de Componentes – Sistema de Aprendizaje de Armonía Musical con un agente de IA
MusicAI



4. Diagrama de despliegue

Link: [Diagrama de despliegue](#)

Diagrama de Despliegue – Sistema de Aprendizaje de Armonía Musical con un agente de IA
MusicAI



Conclusiones

La definición de la arquitectura de software permitió establecer una estructura organizada, modular y escalable para el sistema **MusicAI**, facilitando la separación de responsabilidades entre frontend, backend, inteligencia artificial, reconocimiento musical y base de datos.

La implementación de patrones de diseño como *MVC*, *Singleton*, *Factory Method*, *Observer*, *Strategy* y *Facade* contribuirá a mejorar la mantenibilidad, reutilización y flexibilidad del sistema durante futuras etapas de desarrollo y evolución tecnológica.

El uso de tecnologías modernas como *Flutter*, *FastAPI*, *SQL Server*, *Python*, *Docker* y *Linux* proporciona una base sólida para el desarrollo de una aplicación móvil robusta, orientada al aprendizaje musical interactivo y procesamiento inteligente de información.

La incorporación de procesamiento digital de señales (**DSP**) permitirá realizar reconocimiento y análisis musical en tiempo real, mejorando la precisión de validación de ejercicios prácticos ejecutados por el usuario.

La arquitectura propuesta contempla un enfoque orientado a inteligencia artificial propia, permitiendo entrenar modelos especializados en teoría musical y armonía funcional utilizando bases de conocimiento del sistema **MusicAI**.

El **diagrama de componentes** permitió representar la organización lógica del software y la interacción entre los diferentes módulos internos del sistema, facilitando la comprensión de dependencias y servicios principales.

El **diagrama de despliegue** permitió representar la arquitectura física de implementación del sistema, identificando dispositivos, servidores, artefactos, almacenamiento y relaciones de comunicación necesarias para el funcionamiento de **MusicAI**.

El desarrollo de esta evidencia permitió fortalecer el análisis arquitectónico del proyecto, estableciendo una base técnica coherente para futuras fases de implementación, pruebas y despliegue del sistema **MusicAI**.